

LOGBOOK OF FMU_explore

DEVELOPMENT

Here I describe the status of development the module FMU_explore. The code has been developed over long time as a gernal part of a setup script for BPL applications. Now the time has come to encapsulate this code as a central module. The simplified FMU interaction can be used for Modelica simulation models in general and not just BPL applications.

This informal text briefly describes the status and goals in near time. The idea is that this text here will complement the formal git-version handling. Now and then I plan to introduce reference to the git-version commit history.

Key architecture features are:

- Class structure of the module
- The module is then adapted to an applications through a number of variables
- The module placed in a package to simplify future use both locally and in Github

Development is mainly done in the following files:

- fmu_explore_pyfmi.py
- fmu_explore_fmppy.py
- setup TEST2_Batch.py
- setup TEST2_Batch_explore.py

I think the different examples serve as a good test bench for changes in the library. The collection of processes also is a good starting point when new systems are developed.

A general goal is:

Specific goals - now reached:

- Module v1 without simu(), show(), setLines() / Done 260819
- Module v2 complete a new concept axes as a list / Done 260819
- Module v2 now works in Windows with PyFMI / Done 260820
- Module v2 now works in Linux with PyFMI and no surprises / Done 260821
- Module v2 stored in package and used both locally and in Colab / Done 260824
- How do I make the performance function cstrProdMax() available inside simu() / Done

260825

- Make `sim_res` available in the Jupyter/Colab notebook when needed / Done 260829
- Make `simulationTime` default for `simu()` and also possible to set in `simu()` / Done 260901
- Make options in `simu()` possible to set / Done 260901
- Make `cstrProdMax()` available during plotting / Done 260901
- Make `profile()` available during plotting / Done 260901
- Fix `simu('cont')` for `fmu_explore_fmpy` and `model_get()` / Done 260907
- Make declaration of version in only one place from today's two places / Done 260909
- Made `prevFinalTime` local to the `FMU_explore` class / Done 260909
- Added to `simu()` the context parameter with `model_get()` good for `eval()` / Done 260909
-
- **Make version of `scipy.optimize` visible in `system_info()`**
- Reorganise the list of arguments for the class (and update notebooks)
- Handle `cstrProdMax` etc for `fmpy`

Day to day notes in reversed time order

Note that commit ID are now given after the date and description (i.e. opposite to Git). Far from all commits are described here though.

2026-09-08 Fixed in `fmu_explore_fmpy/decribe_general()` a `model.get` to `model_get`. Call it now **ver 1.1.6**.

2026-09-08 Now `BPL_TEST2_Fedbatch` and `BPL_YEAST_AIR_Fedbatch_DOcontrol` works with `fmpy` in ver 1.1.5.

2026-09-07 Now `BPL_TEST2_Batch` works with `fmuexplorefmpy` in **ver 1.1.5**. This version of `FMU_explore` will be uploaded to Github.

2026-09-02 Start the work with `fmu_explore_fmpy` and call it ver 1.1.5

2026-09-01 Brought in the liast three applictions `Chemostat`, `Perfusion` and `IEC_validation` and using **FMU_explore ver 1.1.4**.

2026-09-01 Brought in `BPL_TEST2_Batch_calibration`.

2026-09-01 I made `simu()` have parameters `simulationTime` and options free to set but default taken from the class. Call it **FMU_explorer ver 1.1.3**.

2026-08-28 Now brought in `BPL_YEAST_COB_Batch` and `BPL_TEST2_Batch_design_space` using `FMU_explore` ver 1.1.2

2026-08-29 I made `sim_res` available outside `simu()` by just introducing a return statement in the function. Call this **FMU_explorer ver 1.1.2**.

2026-08-28 Now brought in BPL_IEC_Operation, BPL_STEM_AIR_Perfusion.

2026-08-27 Now brought in BPL_CHO_Perfusioncspropenloop.

2026-08-26 Now brought three more applications to Colab: PL_TEST2_Chemostat, BPL_YEAST_AIR_Fedbatch, BPL_CHO_Fedbatch.

2026-08-25 Almost managed to convert BPL_TEST2_Chemostat for FMU_explore 1.1.0. Remains to handle cstrProdMax() defined in user functions, though.

2026-08-24 The package updated to include import of pandas. Then run locally with BPL_TEST2_Batch, BPL_TEST2_Fedbatch and BPL_CHO_Fedbatch in Windows. The next step is to run these applications also in Colab after updating the Colab notebook. Problem today with pyfmi more general.

2026-08-21 I tested the application BPL_TEST2_Batch in Linux and worked fine. Then I made it public at Github Colab and published a note as well.

2026-08-20 I decided to let the itertool work from the module only. The function resetPen() can be used in the function newplot() in the applications script and that is the key. Also the sim_res file is for the time being just kept global from the module.

2026-08-19 Managed to get v2 working reasonably. The key was to introduce an empty list ax, similar to diagrams, as part of the class. Then newplot() gives both of them appropriate values. Remains to fix setLines() and also simu() bring back start and final simulation time.